JPA query creation

학습목표

JPA Repository 쿼리 메소드 자동 구현 기능
JPA Repository 쿼리 메소드 이름 규칙

목차

1. 개요 2
    1) @Table, @Column 2
    2) Spring Data JPA 3
    3) JPA Repository 기본 메소드 3
    4) 쿼리 메소드 자동 구현 (query creation) 4
    5) 쿼리 메소드 이름 규칙 6
    6) countBy, deleteBy, removeBy 11
2. jpa5 프로젝트 12
    1) 프로젝트 생성 12
    2) build.gradle 파일 수정 14
    3) application.properties 14
    4) Department.java 15
    5) Student.java 15
    6) StudentRepository.java 16
    7) StudentController.java 17
    8) common.css 19
    9) index.html 20
    10) student/list.html 21
    11) 실행 22
3. 연습문제 23

1.개요

1) @Table, @Column

JPA 엔터티 클래스 이름과 DB 테이블 이름이 일치해야 한다.
다행히 Windows mysql은 대소문자에 관대해서, 대소문자가 일치하지 않아도 되지만,
linux mysql은 대소문자까지 일치해야 한다.

student2 데이터베이스의 학생 테이블은 student 이고
학생 엔터티 클래스 이름은 Student 이다.
대소문자가 다르기 때문에 linux mysql에서 실행하면 에러가 발생한다.

만약 이름이 일치하지 않은 경우에,
@Table(name="테이블이름") 어노테이션을 사용해서 테이블 이름을 명시해 줄 수 있다.

JPA 엔터티 클래스의 속성 이름이 DB 테이블의 필드 이름과 일치해야 한다.
만약 이름이 일치하지 않은 경우에,
@Column(name="필드이름") 어노테이션을 사용해서 필드 이름을 명시해 줄 수 있다.

```
package net.skhu.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

import lombok.Data;

@Data
@Entity
@Table(name="department")
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    @Column(name="title")
    String name;
}
```

```
@Table(name="department")
public class Department {
   이 엔터티 클래스와 연결되는 테이블의 이름은 "department" 이다.
   리눅스 mysql은 대소문자까지 일치해야 하므로, 이렇게 적어주어야 한다.

@Column(name="title")
String name;
   이 속성과 연결될 필드의 이름은 "title" 이다.
```

2) Spring Data JPA

Spring Data는 Spring 프로젝트에서 사용되는 데이터베이스 라이브러리 개발 프로젝트이다.
https://spring.io/projects/spring-data

Spring Data JDBC 라이브러리에는
JDBC 데이터베이스 프로그래밍을 편하게 할 수 있도록 도와주는 클래스들이 포함되어 있다.

Spring Data JPA 라이브러리에는
JPA 데이터베이스 프로그래밍을 편하게 할 수 있도록 도와주는 클래스들이 포함되어 있다.

JPA 프로젝트와 Spring Data JPA 프로젝트가 서로 다른 프로젝트다.

우리가 예제에서 사용하고 있는 JpaRepository 인터페이스는
JPA에 포함된 것이 아니고 Spring Data JPA에 포함되어 있다.

이 수업에서 우리가 배우고 있는 JPA 프로그래밍은
정확히 말하자면 JPA 기술이 아니고 Spring DATA JPA 기술이다.

3) JPA Repository 기본 메소드

내가 사용하고 있는 클래스가 어떤 라이브러리 소속인지는 패키지 이름을 보면 알 수 있다.

```
package net.skhu.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import net.skhu.entity.Department;

public interface StudentRepository extends JpaRepository<Student, Integer>  {

}
```

우리가 JpaRepository 인터페이스를 상속하여 StudentRepository 인터페이스를 정의하면,
Spring Data JPA가 다음과 같은 일들을 자동으로 해준다.
- StudentRepository 인터페이스를 구현한 클래스를 자동으로 구현한다.
- 그 클래스에, JPA repository 기본 메소드(findById, findAll, save, delete, exists)를 구현해 준다.
- 그 클래스의 객체를 한 개 자동 생성해 준다.
- 생성된 객체를 @Autowired StudentRepository studentRepository; 멤버 변수에 자동으로 대입해 준다.

4) 쿼리 메소드 자동 구현 (query creation)

Spring Data JPA는, JPA repository 기본 메소드 뿐만 아니라,
repository 인터페이스에 선언된 abstract 메소드들도 자동으로 구현해 준다.

예:

```
public interface StudentRepository extends JpaRepository<Student, Integer>  {

    List<Student> findByName(String name);
    Student        findByStudentNo(String studentNo);
    List<Student> findByNameStartsWith(String name);
    List<Student> findByDepartmentName(String name);
    List<Student> findByDepartmentNameStartsWith(String name);

    List<Student> findAllByOrderByName();
    List<Student> findAllByOrderByNameDesc();
    List<Student> findByDepartmentIdOrderByNameDesc(int id);
}
```

위 메소드 이름에서
노란색으로 칠한 name, department 부분은 Student 엔터티 클래스의 속성의 이름이다.
Student 엔터티 클래스에 department 속성이 정의되어 있다.

Student 엔터티 클래스의의 name 속성, department 속성은 소문자로 시작하지만,
JPA 메소드에 속성명 부분은 반드시 대문자로 시작해야 한다.

Java에서 속성은 멤버 변수가 아니고, getter/setter 메소드에 의해서 정의된다.
Student 엔터티 클래스의 getter/setter 메소드는, lombok에 의해서 자동 생성된다.

하늘색으로 칠한 부분은 Department 엔터티 클래스의 속성 이름이다.
Student 테이블과 Department 테이블은 departmentId 외래키로 조인된다.
조인된 테이블의 속성임을 명시적으로 표현하기 위해서,
즉 Student의 속성이 아니고 Department의 속성임을 명시적으로 표현하기 위해서,
    _ 문자를 넣을 수도 있다. (생략해도 된다)

예) findByDepartment_Name(String name);
    findByDepartment_NameStartWith(String name);
    findByDepartment_IdOrderByNameDesc(int id);

List<Student> findByName(String name);
    student 테이블에서 name 필드로 레코드를 조회하여 리턴한다.
    파라미터 변수 name의 값과 필드 값 전체가 일치해야 한다.

List<Student> findByStudentNo(String studentNo);
    student 테이블에서 studentNo 필드로 레코드를 조회하여 리턴한다.
    파라미터 변수 studentNo의 값과 필드 값 전체가 일치해야 한다.

List<Student> findByNameStartsWith(String name);
    student 테이블에서 name 필드로 레코드를 조회하여 리턴한다.
    파라미터 변수 name의 값이 필드 값 앞 부분과 일치하면 된다.
    예를 들어 파라미터 변수 name의 값이 "김" 이면, 성이 김씨인 레코드 전체를 조회한다.

List<Student> findByDepartmentName(String name);
    student 테이블과 department 테이블을 조회하고,
    조인된 department 테이블의 name 필드로 조회하여 일치하는 student 테이블 레코드를 리턴한다.
    파라미터 변수 name의 값과 department.name 필드 값 전체가 일치해야 한다.

List<Student> findByDepartmentNameStartsWith(String name);
    위 findByDepartmentName 메소드의 설명과 동일하다.
    단 파라미터 변수 name의 값이, department.name 필드 값 앞 부분과 일치하면 된다.

List<Student> findAllByOrderByName();
    student 테이블의 전체 레코드를, 필드의 오름차순으로 정렬하여 리턴한다.
    위 메소드 이름에서 All은 생략할 수 있다. By는 생략할 수 없다.

```
List<Student> findAllByOrderByNameDesc();
    student 테이블의 전체 레코드를, 필드의 내림차순으로 정렬하여 리턴한다.

List<Student> findByDepartmentIdOrderByNameDesc(int id);
    student 테이블과 department 테이블을 조회하고,
    조인된 department 테이블의 id 필드로 조회하여 일치하는 student 테이블 레코드를 리턴한다.
    조회 결과를 student 테이블의 name 필드의 내림차순으로 정렬하여 리턴한다.


위 조회 메소드들의 파라미터 변수 이름은 전혀 중요하지 않다. 메소드 이름만 중요하다.
  예: List<Student> findByOrderByName();
            findBy 뒤에 조회 조건이 없으므로 파라미터 변수도 없다.
      List<Student> findByStudentNo(String sno);
            findBy 뒤에 조회 조건이 한 개이므로 파라미터 변수도 한 개다. 변수 이름은 아무거나 상관없다.
      List<Student> findByStudentNoOrName(String sno, String nm);
            findBy 뒤에 조회 조건이 두 개이므로 파라미터 변수도 두 개다. 변수 이름은 아무거나 상관없다.
            예를 들어 studentRepository.findByStudentNoOrName("201514099", "홍길동");
            이렇게 호출하면 학번이 "201514099" 이거나 이름이 "홍길동"인 학생들을 조회한다.
```

5) 쿼리 메소드 이름 규칙

Person 엔터티 클래스의 속성

```
String firstName
String lastName
Date startDate
Integer age
boolean active
```

Supported keywords inside method names

Keyword	Sample
And	findByLastNameAndFirstName(String lastName, String firstName)
Or	findByLastNameOrFirstName(String lastName, String firstName)
Between	findByStartDateBetween(Date date1, Date date2)
LessThan	findByAgeLessThan(int age)
GreaterThan	findByAgeGreaterThan(int age)
LessThanEqual	findByAgeLessThanEqual(int age)
GreaterThanEqual	findByAgeGreaterThanEqual(int age)
After	findByStartDateAfter(Date date1)
Before	findByStartDateBefore(Date date1)
IsNull	findByAgeIsNull()
IsNotNull,NotNull	findByAgeNotNull(), findByAgeIsNotNull()
Like	findByFirstNameLike(String pattern) pattern값의 예: "김%" "%홍%", "%진"
NotLike	findByFirstNameNotLike(String pattern)
StartingWith	findByFirstNameStartingWith(String name) name 값의 예: "김"
EndingWith	findByFirstNameEndingWith(String name) name 값의 예: "진"
Containing	findByFirstNameContaining(String name) name 값의 예: "홍"
OrderBy	findByAgeOrderByLastNameDesc(int age)
Not	findByLastNameNot(String name)
In	findByAgeIn(Collection<Integer> ages)
NotIn	findByAgeNotIn(Collection<Integer> age)
True	findByActiveTrue()
False	findByActiveFalse()
Top	findTop10ByOrderByAge() 조회 결과의 선두 10개만 리턴한다

파라미터 변수 이름은 중요하지 않다. 파라미터 변수 순서가 중요하다.

예:
   findByLastNameAndFirstName(String s1, String s2)
   파라미터 변수 이름이 무엇이든 상관 없이
   첫째 파라미터가 lastName이고 둘째 파라미터가 firstName 이어야 한다.

쿼리 메소드 이름 규칙의 예

Person 테이블
   firstName   NVARCHAR(50)
   lastName   NVARCHAR(50)
   startDate   DATE
   age   int
   active   BOOLEAN

위 표의 샘플 메소드를 SQL 명령으로 표현하자.
메소드 파라미터는 mybatis 파라미터 형식으로 표현함.

```
findByLastNameAndFirstName(String lastName, String firstName)
    SELECT * FROM Person WHERE lastName = #{lastName} AND firstName = #{firstName}

findByLastNameOrFirstName(String lastName, String firstName)
    SELECT * FROM Person WHERE lastName = #{lastName} OR firstName = #{firstName}

findByStartDateBetween(Date date1, Date date2)
    SELECT * FROM Person WHERE startDate BETWEEN #{date1} AND #{date2}

findByAgeLessThan(int age)
    SELECT * FROM Person WHERE age < #{age}

findByAgeGreaterThan(int age)
    SELECT * FROM Person WHERE age > #{age}

findByStartDateAfter(Date date1)
     SELECT * FROM Person WHERE startDate > #{date1}

findByStartDateBefore(Date date1)
     SELECT * FROM Person WHERE startDate < #{date1}

findByAgeIsNull()
     SELECT * FROM Person WHERE age IS NULL

findByAgeNotNull(), findByAgeIsNotNull()
     SELECT * FROM Person WHERE age IS NOT NULL


findByFirstNameLike(String pattern)
     SELECT * FROM Person WHERE firstName LIKE #{pattern}

findByFirstNameNotLike(String pattern)
     SELECT * FROM Person WHERE firstName NOT LIKE #{pattern}

findByFirstNameStartingWith(String name)
findByFirstNameStartsWith(String name)
     SELECT * FROM Person WHERE firstName LIKE #{name} + '%'
     예) SELECT * FROM Person WHERE firstName LIKE '김%'

findByFirstNameEndingWith(String name)
findByFirstNameEndsWith(String name)
     SELECT * FROM Person WHERE firstName '%' + LIKE #{name}
     예) SELECT * FROM Person WHERE firstName LIKE '%김'
```

```
findByFirstNameContaining(String name)
findByFirstNameContains(String name)
     SELECT * FROM Person WHERE firstName '%' + LIKE #{name} + '%'
     예) SELECT * FROM Person WHERE firstName LIKE '%김%'

findByAgeOrderByLastNameDesc(int age)
     SELECT * FROM Person WHERE age = #{age} ORDER BY lastName DESC

findByAgeOrderByLastNameAscFirstNameAsc(int age)
     SELECT * FROM Person WHERE age = #{age} ORDER BY lastName ASC, firstName ASC

findByLastNameNot(String name)
     SELECT * FROM Person WHERE lastName <> #{name}

findByAgeIn(Collection<Integer> ages)
     SELECT * FROM Person WHERE age IN #{ages}
     age 필드 값이 ages 목록에 들어있는 레코드만 조회
     /* 주의: mybatis는 Collection 타입 파라미터를 지원하지 않음 */

findByAgeNotIn(Collection<Integer> age)
     SELECT * FROM Person WHERE age NOT IN #{ages}
     /* 주의: mybatis는 Collection 타입 파라미터를 지원하지 않음 */

findByActiveTrue()
     SELECT * FROM Person WHERE active
     SELECT * FROM Person WHERE active = true

findByActiveFalse()
     SELECT * FROM Person WHERE NOT active
     SELECT * FROM Person WHERE active = false

findTop10ByOrderByAge()
     SELECT TOP 10 * FROM Person ORDER BY age (SQL Server 문법)
     SELECT * FROM Person ORDER BY age LIMIT 10 (MySQL 문법)
     SELECT * FROM Person WHERE ROWNUM <= 10 ORDER BY age (Oracle 문법)
     조회 결과에서 선두 10 개 레코드만 리턴한다.


repository 인터페이스에 선언된 메소드의 이름이 위 규칙에 부합하면,
그 메소드는 Spring Data JPA 에 의해서 자동 구현된다.
```

리턴 타입

findBy 메소드의 리턴 타입을, List<Student> 타입으로 선언해도 되고,
Student 타입으로 선언해도 된다.

List<Student> findByName(String name);
  조회 결과 레코드가 0 개 이라면 비어있는 List<Student> 객체를 리턴한다.
  조회 결과 레코드 수가 몇 개이든지 에러가 발생하지 않는다.

Student findByName(String name);
  조회 결과 레코드가 0 개 이면 null 을 리턴한다.
  조회 결과 레코드가 1 개 이면 Student 객체를 리턴한다.
  만약 조회 결과가 레코드 두 개 이상이라면 에러가 발생한다.

Optional<Student> findByName(String name);
  Optional 클래스는 택배 박스와 비슷한 역할을 하는 객체이다.
  위 메소드는 Optional 상자 안에 Student 객체를 넣어서 리턴한다.

  조회 결과 레코드가 0 개 이면 속이 빈 Optional 객체를 리턴한다.
  조회 결과 레코드가 1 개 이면 속에 Student 객체가 들어있는 Optional 객체를 리턴한다.
  만약 조회 결과가 레코드 두 개 이상이라면 에러가 발생한다.

에러 메시지

만약 메소드 이름이 규칙에 어긋난다면,
STS 편집창에서 에러 메시지가 표시될 것이다.

예를 들어, Student 엔터티의 name 속성명을 namme 으로 수정하자.
실행을 시작할 때, 아래와 같은 에러 메시지가 표시된다.

```
"Invalid derived query! No property namme found for type Student! Did you mean 'name'?"
```

메소드 이름의 다른 부분에 오타를 만들고, 어떤 에러 메시지가 표시되는지 확인하자.
고수가 되려면, 에러 메시지만 보고 원인을 짐작할 수 있어야 한다.
에러 메시지에 익숙해져야 한다.

findAllBy### 와 findBy###

메소드 이름에서 find와 By 사이에 All을 넣어도 좋고 생략해도 좋다.

6) countBy, deleteBy, removeBy

findBy 메소드 이름 규칙은
countBy, deleteBy, removeBy 메소드에도 적용된다.

countBy 메소드는 주어진 조건에 일치하는 레코드 수를 리턴한다.

deleteBy, removeBy 메소드는 주어진 조건에 일치하는 레코드를 삭제한다.

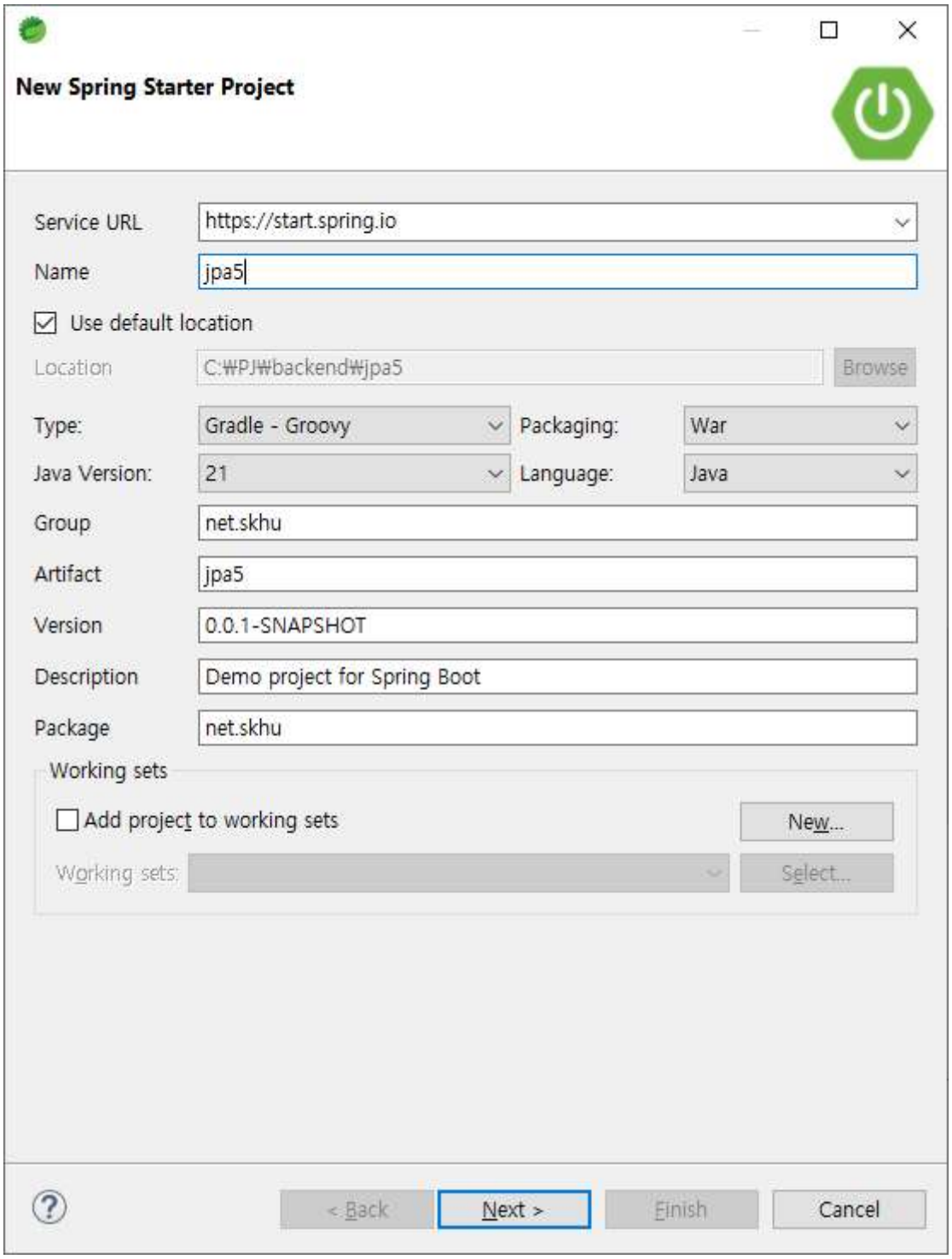
예:

```
public interface StudentRepository extends JpaRepository<Student, Integer>  {
    int countByName(String name);
    int countByNameStartsWith(String name);
    int countByStudentNoOrName(String studentNo, String name);

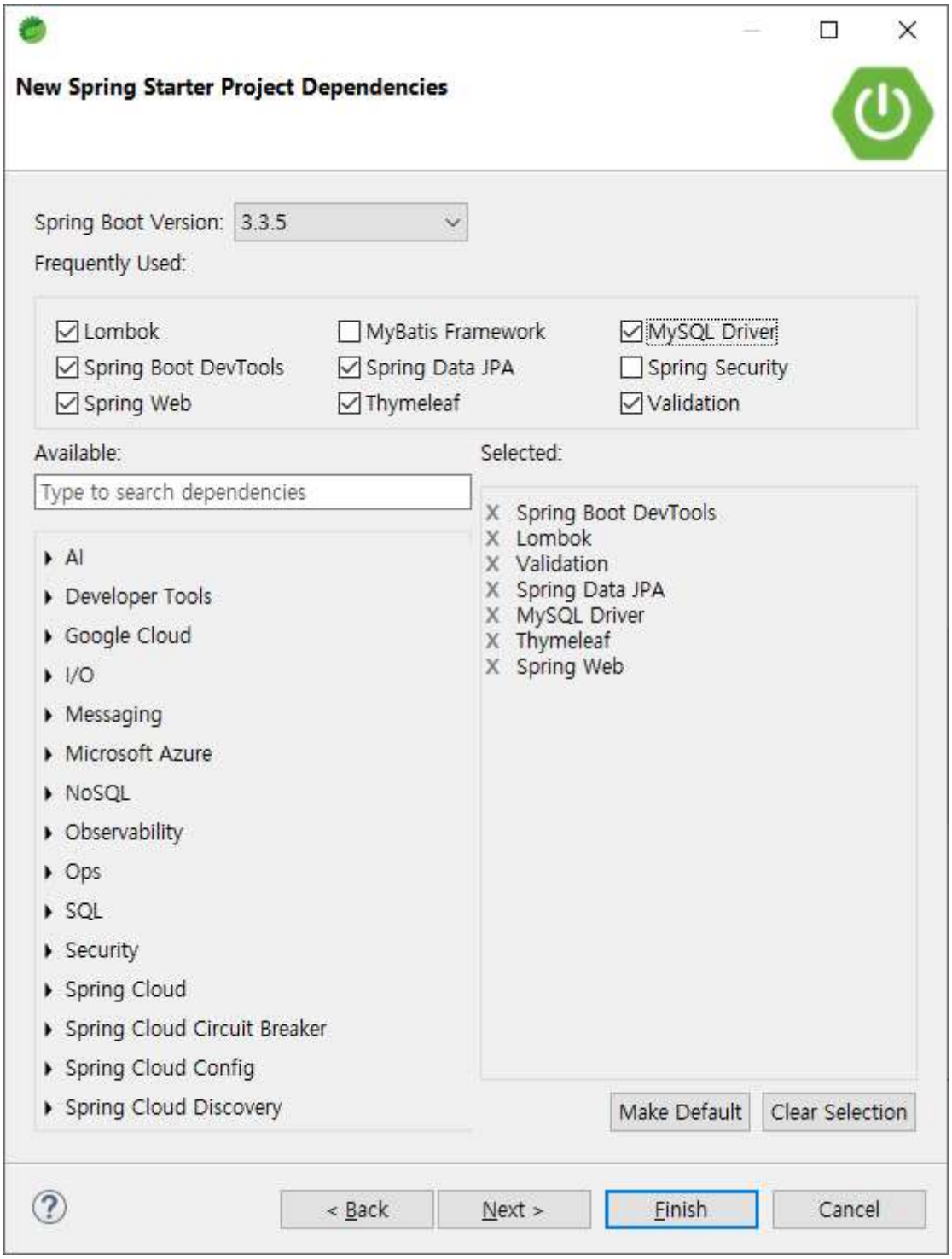
    void deleteByName(String name);
    void deleteByDepartmentId(int departmentId);
}
```

2.jpa5 프로젝트

1) 프로젝트 생성



Name	jpa5
Type	Gradle - Groovy
Package	net.skhu




```
Spring Boot DevTools
Lombok
Validation
Spring Data JPA
MySQL Driver
Thymeleaf
Spring Web
```

2) build.gradle 파일 수정

```
dependencies {
  ... 생략 ...
    implementation 'org.modelmapper:modelmapper:3.2.5'
}
```

gradle 프로젝트의 설정 파일인 build.gradle 파일에
  modelmapper maven dependency 한 줄을 추가

Referesh Gradle Project 메뉴 실행
  build.gradle 파일을 수정한 후에는 반드시 Refresh Gradle Project 해야 한다.

3) application.properties

```
spring.application.name=jpa5
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/student2?useUnicode=yes&characterEncoding=UTF-8&allowMultiQueries=true&serverTimezone=Asia/Seoul
spring.datasource.username=user1
spring.datasource.password=skhuA+4.5
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.orm.jdbc.bind=TRACE
logging.level.net.skhu=DEBUG
server.port=8088
```

4) Department.java

src/main/java/net/skhu/entity/Department.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Data;

@Data
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String shortName;
    String phone;
}
```

5) Student.java

src/main/java/net/skhu/entity/Student.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Data;

@Data
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String studentNo;
    String name;
    String phone;
    String sex;
    String email;

    @ManyToOne
    @JoinColumn(name="departmentId")
    Department department;
}
```

6) StudentRepository.java

src/main/java/net/skhu/repository/StudentRepository.java

```
1  package net.skhu.repository;
2
3  import java.util.List;
4  import org.springframework.data.jpa.repository.JpaRepository;
5  import net.skhu.entity.Student;
6
7  public interface StudentRepository extends JpaRepository<Student, Integer>  {
8
9      Student findByStudentNo(String studentNo);
10     List<Student> findByName(String name);
11     List<Student> findByNameStartsWith(String name);
12     List<Student> findByDepartmentName(String name);
13     List<Student> findByDepartmentNameStartsWith(String name);
14
15     List<Student> findByOrderByName();
16     List<Student> findByOrderByNameDesc();
17     List<Student> findByDepartmentIdOrderByNameDesc(int id);
18
19 }
```

7) StudentController.java

src/main/java/net/skhu/controller/StudentController.java

```java
package net.skhu.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import net.skhu.entity.Student;
import net.skhu.repository.StudentRepository;

@Controller
public class StudentController {

    @Autowired StudentRepository studentRepository;

    @GetMapping("/")
    public String index() {
        return "index";
    }

    @GetMapping("student/test1")
    public String test1(Model model, @RequestParam("studentNo") String studentNo) {
        Student student = studentRepository.findByStudentNo(studentNo);
        model.addAttribute("students", student);
        return "student/list";
    }

    @GetMapping("student/test2")
    public String test2(Model model, @RequestParam("name") String name) {
        model.addAttribute("students",
                studentRepository.findByName(name));
        return "student/list";
    }

    @GetMapping("student/test3")
    public String test3(Model model, @RequestParam("nm") String name) {
        model.addAttribute("students",
                studentRepository.findByNameStartsWith(name));
        return "student/list";
    }

    @GetMapping("student/test4")
    public String test4(Model model, @RequestParam("dn") String departmentName) {
        model.addAttribute("students",
                studentRepository.findByDepartmentName(departmentName));
        return "student/list";
    }

    @GetMapping("student/test5")
    public String test5(Model model, @RequestParam("dn") String departmentName) {
        model.addAttribute("students",
                studentRepository.findByDepartmentNameStartsWith(departmentName));
        return "student/list";
    }

    @GetMapping("student/test6")
    public String test6(Model model) {
        model.addAttribute("students",
                studentRepository.findByOrderByName());
        return "student/list";
    }

    @GetMapping("student/test7")
    public String test7(Model model) {
        model.addAttribute("students",
                studentRepository.findByOrderByNameDesc());
        return "student/list";
    }
```

```
69
70      @GetMapping("student/test8")
71      public String test8(Model model, @RequestParam("di") int departmentId) {
72          model.addAttribute("students",
73                  studentRepository.findByDepartmentIdOrderByNameDesc(departmentId));
74          return "student/list";
75      }
76  }
```

8) common.css

src/main/resources/static/common.css

```
div.container { width: 800px; margin: 10px auto; font-size: 11pt; }
.shadow { box-shadow: 2px 2px 5px gray; }

.btn { padding: 0.4em 1em; border: 1px solid gray;
       border-radius: 0.5em; background: linear-gradient(#fff, #ddd);
       text-decoration: none; color: black;
       display: inline-block; }
.btn:active {
      -ms-transform: translateY(2px);
      -webkit-transform: translateY(2px);
      transform: translateY(2px);
      background: #ccc; }

table.list { border-collapse: collapse; width: 100%; }
table.list td { padding: 4px; border: 1px solid gray; }
table.list th { padding: 4px; border: 1px solid gray;  background-color: #eee; }

div.row { margin-bottom: 0.8em; }
div.row > label:nth-child(1) { display: inline-block; min-width: 4em; text-align: right; }

input[type=text], input[type=number] { padding: 0.3em; }
select { padding: 0.3em; }
h1 { margin: 0.5em 0; }
hr { margin: 1em 0; }
.error { color: red; padding: 0.2em; margin-left: 1em; }

div.pagination table { border-collapse: collapse; margin-top: 1em; }
div.pagination table td { border: 1px solid #ccc; text-align: center; }
div.pagination table td.active { background-color: #48f; }
div.pagination table a { text-decoration: none; color: blue; display: inline-block;
                         width: 25px; padding: 6px; }
div.pagination table td.active a { color: white; }
```

9) index.html

src/main/resources/templates/index.html

```
<!doctype html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
</head>
<body>
<div class="container">
  <h1>index</h1>
  <ul>
    <li><a href="student/test1?studentNo=201532011">test1?studentNo=201532011</a></li>
    <li><a href="student/test2?name=최하은">test2?name=최하은</a></li>
    <li><a href="student/test3?nm=이">test3?nm=이</a></li>
    <li><a href="student/test4?dn=소프트웨어공학과">test4?dn=소프트웨어공학과</a></li>
    <li><a href="student/test5?dn=소">test5?dn=소</a></li>
    <li><a href="student/test6">test6</a></li>
    <li><a href="student/test7">test7</a></li>
    <li><a href="student/test8?di=1">test8?di=1</a></li>
  </ul>
</div>
</body>
</html>
```

URL에 한글 문자

원래 URL에 한글 문자를 사용할 수 없다.
URL에 한글은 %EC%B5%9C%ED%95%98%EC%9D%80 형태로 url encoded 되어야 한다.

예를 들어 student/test2?name=최하은 URL은
student/test2?name=%EC%B5%9C%ED%95%98%EC%9D%80 이어야 한다.

그런데 요즘 웹브라우저들은 URL에 한글 문자 사용을 허용한다.
한글 문자가 포함된 URL을 웹브라우저가 서버에 요청할 때,
자동으로 URL의 한글을 %EC%B5%9C%ED%95%98%EC%9D%80 형태로 url encoding 해주기 때문이다.

query string name과 request parameter 이름

URL의 query string이 request parameter가 되어 서버의 액션 메소드에 전달된다.

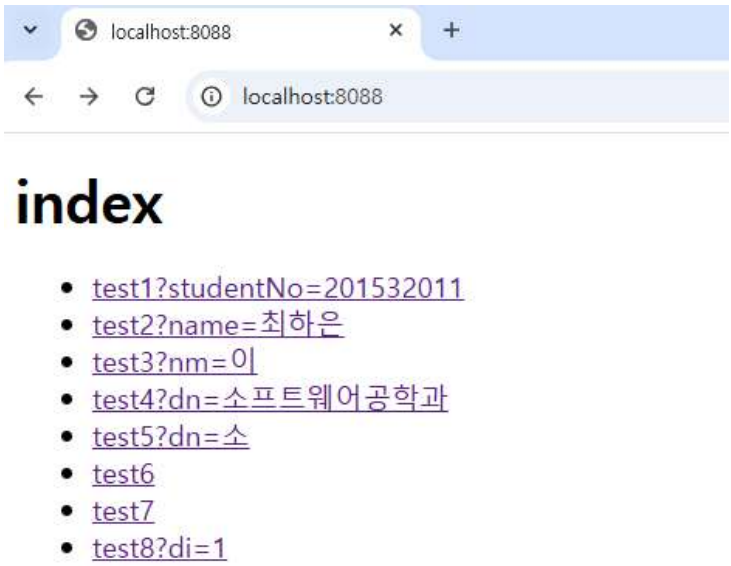
위 URL의 query string 이름과, 컨트롤러 액션 메소드의 request param 이름이 일치하는지 비교해보라.
만약 위 이름이 서로 일치하지 않으면 어떤 에러가 발생하는지 꼭 확인해 보라.

10) student/list.html

src/main/resources/templates/student/list.html

```
<!doctype html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
</head>
<body>
<div class="container">
  <h1>학생목록</h1>
  <table class="list">
    <thead>
      <tr>
        <th>id</th>
        <th>학번</th>
        <th>이름</th>
        <th>학과</th>
        <th>전화</th>
        <th>성별</th>
        <th>이메일</th>
      </tr>
    </thead>
    <tbody>
      <tr th:each="st : ${ students }">
        <td th:text="${ st.id }"></td>
        <td th:text="${ st.studentNo }"></td>
        <td th:text="${ st.name }"></td>
        <td th:text="${ st.department.name }"></td>
        <td th:text="${ st.phone }"></td>
        <td th:text="${ st.sex }"></td>
        <td th:text="${ st.email }"></td>
      </tr>
    </tbody>
  </table>
</div>
</body>
</html>
```

11) 실행


localhost:8088
localhost:8088
index
test1?studentNo=201532011
test2?name=최하은
test3?nm=이
test4?dn=소프트웨어공학과
test5?dn=소
test6
test7
test8?di=1

3. 연습문제

JPA query creation 기능으로 다음 기능들을 구현해보라

1) department id 값이 3, 4 인 학생들만 조회하기

2) 김씨 이거나 박씨인 학생들 조회하기

3) 김씨 학생을 이름 오름차순으로 10명만 조회하기

4) 김씨인 소프학생들만 이름 오름차순으로 조회하기